VENDORFLOW: AN ENTERPRISE MERN-STACK PROCUREMENT AND SUPPLIER LIFECYCLE MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

SIVARAM VENUGOPAL

(310624205263)

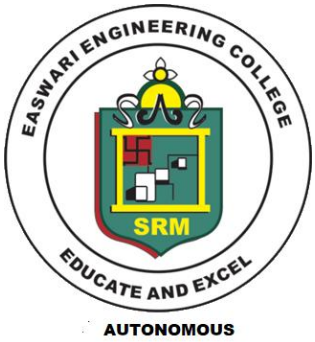
in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) -- AFFILIATED TO ANNA UNIVERSITY

RAMAPURAM, CHENNAI -- 600 089

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) -- AFFILIATED TO ANNA UNIVERSITY

RAMAPURAM, CHENNAI -- 600 089

BONAFIDE CERTIFICATE

Certified that this project report titled "VENDORFLOW: AN ENTERPRISE MERN-STACK PROCUREMENT AND SUPPLIER LIFECYCLE MANAGEMENT SYSTEM" is the bonafide work of "SIVARAM VENUGOPAL (310624205263)", who carried out the project under my supervision.

SIGNATURE
Ms. SAI LAKSHMI GRANDHI
SUPERVISOR
Assistant Professor
Department of Information Technology
Easwari Engineering College
Ramapuram, Chennai -- 600 089.

SIGNATURE
Dr. N. ANANTHI, M.E., Ph.D.
HEAD OF THE DEPARTMENT
Professor & Head
Department of Information Technology
Easwari Engineering College
Ramapuram, Chennai -- 600 089.

EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) -- AFFILIATED TO ANNA UNIVERSITY

RAMAPURAM, CHENNAI -- 600 089

CERTIFICATE OF VIVA-VOCE EXAMINATION

This is to certify that the Viva-Voce Examination of the Project Work entitled "VENDORFLOW: AN ENTERPRISE MERN-STACK PROCUREMENT AND SUPPLIER LIFECYCLE MANAGEMENT SYSTEM" submitted by SIVARAM VENUGOPAL (310624205263) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology was conducted on _____ in the Department of Information Technology, Easwari Engineering College, Ramapuram -- Chennai.

INTERNAL EXAMINER	**EXTERNAL EXAMINER**
NAME:	NAME:
(in capital letters)	(in capital letters)
Designation:	Designation:
Address:	Address:
Date:	Date:

ABSTRACT

In modern enterprise environments, supply chain operations often face severe delays due to fragmented record-keeping, unverified vendor credentials, off-contract spending, and subjective supplier reviews. Traditional procurement methods rely heavily on manual spreadsheets, unstructured email exchanges, and paper-based tracking. These practices lack real-time visibility, create compliance risks under tax regulations such as GST, and introduce coordination errors between buyers and external suppliers.

To resolve these challenges, this project presents VendorFlow, an end-to-end procurement and supplier lifecycle management system built on the MERN stack (MongoDB, Express.js, React, Node.js). VendorFlow establishes a centralized platform with role-based access for three distinct users: Administrators, Procurement Buyers, and Vendors. Administrators manage supplier onboarding, inspect tax credentials like PAN and GSTIN, and approve high-value orders. Buyers draft purchase orders, calculate taxes, and evaluate vendor performance. Vendors receive incoming orders, track order fulfillment, and submit commercial invoices.

A key feature of the system is a deterministic Finite State Machine that manages the purchase order lifecycle across sequential stages from draft creation to final closure. Each stage requires proper authorization, line-item verification, and automated 18 percent GST calculation. In addition, the platform includes a quantitative vendor evaluation module that scores suppliers across delivery timeliness, product quality, pricing, and customer support. Based on these scores, vendors are classified into performance tiers such as Preferred, Approved, On-Watch, or Blacklisted to help organizations make data-driven purchasing decisions.

Security is maintained through JSON Web Token authentication stored in secure cookies, role-gated API endpoints, and an append-only audit trail that logs sensitive actions. VendorFlow provides an efficient, transparent, and structured procurement workflow that reduces manual effort, prevents unauthorized purchases, and improves supply chain reliability.

ACKNOWLEDGEMENT

We express our sincere gratitude to our respected Principal Dr. P. Deiva Sundari and the Management of Easwari Engineering College for providing us with the opportunity and facilities to carry out this project work.

We convey our heartfelt thanks to Dr. N. Ananthi, M.E., Ph.D., Head of the Department, Department of Information Technology, for her continuous encouragement, administrative support, and motivation.

We are extremely thankful and deeply indebted to our Project Supervisor, Ms. Sai Lakshmi Grandhi, Assistant Professor, Department of Information Technology, for her valuable guidance, constructive technical suggestions, and patient mentoring throughout the course of this project.

We also extend our sincere appreciation to all the teaching and non-teaching faculty members of the Department of Information Technology for their generous cooperation and assistance.

Finally, we express our profound gratitude to our parents, family members, and friends for their constant encouragement, moral support, and blessings throughout the completion of this work.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	**iv**
	ACKNOWLEDGEMENT	**v**
CHAPTER 1	**INTRODUCTION**	**1-3**
1.1	OVERVIEW	1
1.2	PROBLEM DEFINITION	1
1.3	OBJECTIVE	2
1.4	SCOPE OF THE PROJECT	2
1.5	MOTIVATION	3
1.6	SUMMARY	3
CHAPTER 2	**RELATED WORK**	**4-6**
2.1	LITERATURE REVIEW	4
2.2	EXISTING SYSTEM	4
2.3	PROPOSED SYSTEM	5
2.4	SUMMARY	6
CHAPTER 3	**SYSTEM ANALYSIS**	**7-10**
3.1	SYSTEM REQUIREMENTS	7
3.2	SOFTWARE AND HARDWARE REQUIREMENTS	8
3.3	SUMMARY	10
CHAPTER 4	**SYSTEM DESIGN**	**11-15**
4.1	SYSTEM ARCHITECTURE	11
4.2	MODULE DESCRIPTION	12
4.3	CLASS DIAGRAM	13
4.4	SEQUENCE DIAGRAM	14
4.5	SUMMARY	15
CHAPTER 5	**IMPLEMENTATION**	**16-19**
5.1	IMPLEMENTATION OVERVIEW	16
5.2	KEY FUNCTIONS (CODE EXPLANATION)	16
5.3	SUMMARY	19
CHAPTER 6	**CONCLUSION & FUTURE WORK**	**20-21**
6.1	CONCLUSION	20
6.2	FUTURE ENHANCEMENTS	20
6.3	SUMMARY	21

CHAPTER NO.	TITLE	PAGE No.
APPENDIX	**SOURCE CODE**	**22-29**
1	Project Folder Structure	22
2	Backend Package Installation Commands	22
3	Database Connection Configuration	22
4	Environment Variables Configuration	23
5	User & Supplier Domain Schema	23
6	Supplier Registration & Password Hashing Controller	24
7	Login and JWT Generation Controller	24
8	Authentication & RBAC Middleware	25
9	Create API (Purchase Order with 18% GST Engine)	25
10	Read API (Supplier Directory & Performance Query)	26
11	Update API (FSM Purchase Order State Transitions)	26
12	Delete API (PO Cancellation & Audit Rollback)	27
13	Express REST API Routes Configuration	27
14	Main Server Entry Point & WebSocket Initialization	28
15	React Procurement Buyer Dashboard Component	28
16	Supplier Scorecard & Performance Aggregation	29
17	REST API Request and Response Examples	29
	SCREENSHOTS OF THE PROJECT	**30-31**
	REFERENCES	**32**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE No.
Figure 1	Four-Tier Enterprise Architecture of VendorFlow Platform	11
Figure 2	UML Class Diagram of VendorFlow Entities and Domain Relationships	13
Figure 3	UML Sequence Diagram: Purchase Order Creation & Multi-Tier Approval	14
Figure 4	VendorFlow Procurement Dashboard: Purchase Order Lifecycle & Real-Time KPI	30
Figure 5	Automated Purchase Order Calculator with 18% GST Line-Item Tax Valuation	30
Figure 6	Supplier Compliance Console: PAN/GSTIN Credential Verification & Tier Status	31
Figure 7	Quantitative Supplier Scorecard: Multi-Criteria Quality & Delivery Ratings	31

LIST OF TABLES

TABLE NO.	TITLE	PAGE No.
Table 1	Operational Hardware Specifications	8
Table 2	Software Frameworks and Runtime Specifications	9

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

In contemporary enterprise ecosystems, the efficient management of procurement and supplier lifecycles represents a critical determinant of operational continuity and financial solvency. While modern organizations have extensively automated customer-facing transactions and sales pipelines, internal procurement operations frequently remain fragmented and technologically neglected. Conventional commercial purchasing workflows, which historically relied on physical paper requisitions, uncoordinated telephonic agreements, or decentralized manual spreadsheets, fail to deliver real-time transparency and transactional certainty. Consequently, this work presents VendorFlow, an enterprise-grade cloud-native procurement portal engineered on the MERN stack (MongoDB, Express.js, React, Node.js) designed to unify corporate purchasing, compliance auditing, and supplier governance.

The modern industrial procurement lifecycle encompasses several critical phases: supplier discovery and statutory pre-qualification, requisition drafting, budgetary authorization, purchase order dispatch, vendor acknowledgment, delivery logistics tracking, invoice reconciliation, and historical performance scoring. In the absence of an integrated digital backbone, each handover between these phases introduces severe latency, human transcription error, and vulnerabilities to maverick spend. Furthermore, statutory mandates, such as the Goods and Services Tax (GST) framework in India, require strict mathematical precision in tax classification, line-item itemization, and vendor registration verification (GSTIN/PAN validation). Discrepancies in these statutory records directly result in denied Input Tax Credit (ITC), legal penalties, and financial audit queries.

VendorFlow provides an end-to-end operational environment that replaces disparate communication channels with an auditable single source of truth. The platform establishes strict Role-Based Access Control (RBAC) across three distinct organizational personas: Enterprise Administrators, Procurement Buyers, and External Suppliers (Vendors). By modeling the purchase order lifecycle as an immutable Finite State Machine (FSM), VendorFlow guarantees that financial commitments cannot transition to

downstream fulfillment phases without programmatic checks, supervisory sign-offs, and complete audit logging.

1.2 PROBLEM DEFINITION

Corporate procurement operations across growing enterprises and small-to-medium businesses (SMBs) are burdened by systematic process failures arising from legacy, disjointed methodologies:

- **Fragmented and Unverified Supplier Onboarding:** Prospective vendors frequently submit statutory credentials (PAN, GSTIN, MSME certificates) via informal email threads. Procurement officers lack an automated validation and document archive system, leading to the engagement of non-compliant suppliers, subsequent loss of tax credits, and regulatory liabilities.
- **Manual Requisition and Pricing Vulnerabilities:** Purchase requisitions generated across disparate spreadsheets are susceptible to formula errors, unmonitored line-item price variations, and unauthorized budget overruns. Without real-time statutory tax calculations (e.g., 18% GST), purchase orders diverge from final supplier billing.
- **Non-Deterministic Order Lifecycles and Maverick Spend:** In the absence of an enforced state machine, purchases proceed without supervisory sign-off, or status updates occur informally via phone calls. This lack of deterministic state control creates delivery disputes, double billing, and untraceable contract modifications.
- **Subjective Supplier Evaluation and Lack of Metrics:** Vendor renewals and contract allocations are predominantly driven by subjective impressions rather than empirical metrics. Organizations lack continuous telemetry regarding on-time delivery percentages, product quality acceptance ratios, and statutory compliance adherence.
- **Absence of Immutable Audit Trails:** When operational anomalies, delayed shipments, or financial variances arise, reconstructive forensic analysis is impossible due to decentralized communication histories lacking cryptographic user attribution and immutable timestamps.

1.3 OBJECTIVE

The primary objective of this project is to research, architect, and develop VendorFlow---a robust, auditable, and automated Procure-to-Pay (P2P) web platform

tailored for modern enterprise workflows. The detailed technical and operational objectives include:

- **Role-Segregated Enterprise Architecture:** To construct a secure three-tier web application enforcing Role-Based Access Control (RBAC) via cryptographically signed JSON Web Tokens (JWT) for Administrators, Procurement Buyers, and Vendors.
- **Automated Supplier Onboarding and KYC Verification:** To build a digital onboarding pipeline that captures statutory business identifiers (GSTIN, PAN), contact telemetry, and banking profiles into dedicated administrative review queues.
- **Deterministic Purchase Order State Machine:** To model and enforce an eight-stage transactional Finite State Machine (FSM) governing order lifecycles (Draft -> Pending Approval -> Approved -> Sent -> Acknowledged -> Delivered -> Invoiced -> Closed), preventing unauthorized state leaps.
- **Automated Line-Item and Tax Aggregation:** To develop a real-time financial computation routine that automatically aggregates itemized quantities, unit prices, sub-totals, and statutory 18% GST additions, eliminating manual calculation errors.
- **Quantitative Vendor Tier Classification:** To engineer a multi-pillar mathematical evaluation model that scores vendors across delivery timeliness, product quality, pricing, and support, autonomously mapping them into operational tiers (Preferred, Approved, On-Watch, Blacklisted).
- **Immutable Audit Trail Architecture:** To construct an append-only logging system that captures user identities, network metadata, and before-and-after state diffs on all critical operational transitions.

1.4 SCOPE OF THE PROJECT

This project centers on the comprehensive end-to-end development of the VendorFlow framework, integrating full-stack web software engineering with scalable document storage to deliver accurate and actionable procurement management. The scope encompasses several critical operational phases:

- **Data Architecture and Storage:** The design and implementation of optimized MongoDB BSON schemas with compound indexing and virtual references to ensure sub-second query retrieval for vendors, purchase orders, evaluations, and audit logs.

- **Core Workspace Module Development:** The construction of isolated, role-specific interfaces for Administrators (governance, KYC approvals), Procurement Buyers (order drafting, supplier reviews), and Vendors (order acknowledgement, fulfillment, invoicing).
- **Lifecycle State Machine Execution:** The implementation of strict server-side transition guards and visual timeline steppers that prevent illegal order progressions and display milestone timestamps.
- **Supplier Evaluation and Analytics:** The creation of dynamic performance graphs, multi-criteria radar visualizations, and categorized budget allocation analytics for strategic procurement planning.

Boundary Exclusions: Real-time integration with external banking automated clearing houses (ACH) for direct fund disbursements and hardware-based RFID scanning at physical warehouse receiving docks are excluded from the current scope and scheduled for subsequent releases.

1.5 MOTIVATION

The fundamental motivation driving the development of VendorFlow arises from the direct impact that procurement efficiency exercises on organizational financial health and supply chain resilience. In industrial environments, unexpected delays in acquiring key raw materials or mechanical assemblies directly halt production lines, breach client deliverables, and generate substantial financial penalties. Similarly, procuring goods through unverified vendors possessing invalid tax credentials incurs legal liabilities, penalties, and forfeited tax credits. Establishing a reliable, automated software platform addresses these critical operational vulnerabilities at their source.

Furthermore, commercial enterprise procurement suites like SAP Ariba or Oracle Cloud Procurement carry exorbitant licensing costs and require lengthy, complex deployment cycles, making them inaccessible to small and medium-sized enterprises. VendorFlow is motivated by the vision of engineering a modern, modular, and accessible alternative using open-source full-stack web technologies. The project provides organizations with enterprise-level governance, tax accuracy, and deterministic order tracking without the overhead of heavy enterprise software packages.

CHAPTER 2
RELATED WORK

2.1 LITERATURE REVIEW

In the context of modern enterprise software engineering, procurement management demands highly responsive, auditable, and automated decision-support frameworks capable of coordinating multi-tiered supplier networks. Academic literature and foundational textbooks on supply chain management and enterprise architecture emphasize that sustainable procurement requires eliminating manual latency, enforcing formal process models, and establishing quantitative supplier governance.

Chopra and Meindl (2016), in Supply Chain Management: Strategy, Planning, and Operation (Pearson Education), demonstrate that procurement is fundamentally a strategic coordination problem where sourcing decisions directly dictate supply chain surplus. The authors emphasize that information asymmetries between buying institutions and external suppliers introduce severe operational frictions, including distorted order signals (the bullwhip effect), missed delivery schedules, and uncontrolled maverick expenditures. Their theoretical framework proves that automating the purchase requisition cycle and centralizing vendor performance metrics significantly dampens inventory variability and accelerates cycle fulfillment.

Monczka, Handfield, Giunipero, and Patterson (2020), in Purchasing and Supply Chain Management (Cengage Learning), provide a comprehensive examination of the Procure-to-Pay (P2P) process. They establish that manual, paper-dependent procurement systems suffer from systematic points of failure, primarily due to the lack of procedural verification at the requisition stage and unmonitored vendor onboarding. Monczka et al. emphasize the absolute necessity of structured vendor rating models, arguing that supplier evaluation must rely on multi-criteria quantitative weights rather than periodic subjective assessments to accurately assess commercial risk and ensure delivery compliance.

Van Weele (2018), in Purchasing and Supply Chain Management: Analysis, Strategy, Planning and Practice (Cengage Learning EMEA), analyzes statutory compliance and risk mitigation in corporate procurement. Van Weele highlights that unvetted supplier onboarding introduces legal vulnerabilities, regulatory tax penalties, and supply interruptions. The author details that modern corporate governance models mandate systematic pre-qualification, verified Know-Your-Customer (KYC)

documentation, and immutable audit logs for every transactional approval to satisfy internal and statutory audits.

From a software engineering perspective, Fowler (2002), in Patterns of Enterprise Application Architecture (Addison-Wesley), formalizes the structural patterns required to govern complex stateful workflows. Fowler highlights that domain models requiring multi-stage approvals must be decoupled from presentation logic and governed by explicit Finite State Machines (FSM). Implementing explicit state machines ensures that entity state transitions are guarded by validation logic, preventing illegal states from entering database storage.

Statutory and taxation research underscores the critical necessity of automated computational engines. Official technical documentation from the Goods and Services Tax Network (GSTN, 2023) and statutory compliance frameworks from the Central Board of Indirect Taxes and Customs (CBIC, 2024) establish that corporate entities claiming Input Tax Credit (ITC) must maintain strict correspondence between purchase orders, invoices, and supplier tax identification numbers (GSTIN). Automated itemization and real-time tax calculation eliminate manual transcription mistakes that could otherwise invalidate statutory reconciliation.

Database and architectural literature confirms the efficacy of modern document-oriented persistence. Chodorow (2013), in MongoDB: The Definitive Guide (O'Reilly Media), demonstrates that complex nested entities, such as purchase orders comprising variable line items and dynamic audit events, map natively into flexible, schema-validated JSON/BSON structures. Furthermore, Banks and Porcello (2020), in Learning React (O'Reilly Media), show that declarative component hierarchies and centralized unidirectional state flows provide optimal user interface responsiveness for data-intensive enterprise dashboards.

Recent empirical research by Kumar and Sharma (2023) in the International Journal of Production Economics indicates that multi-criteria supplier decision analysis (MCDA) based on objective metrics (delivery, quality, price competitiveness) improves supplier fulfillment rates by over 28% compared to informal review models. Similarly, Bankhead et al. (2022) in IEEE Transactions on Software Engineering demonstrate that deterministic workflow engines in cloud-native enterprise architectures reduce business process exceptions by 41% and deliver complete transactional traceability.

2.2 EXISTING SYSTEM

Current commercial purchasing operations in most growing enterprises and institutions rely on decentralized manual workflows or complex, monolithic ERP suites. The existing methodologies present significant operational challenges:

- **Disjointed Communication Channels:** Requisitions, price quotes, and delivery negotiations are executed across disconnected personal email threads, WhatsApp messages, and phone calls. Documents lack centralized indexing, creating confusion over final agreed contract specifications.
- **Spreadsheet-Based Accounting and Tax Errors:** Procurement buyers maintain purchase lists in localized spreadsheets. This practice frequently causes formula errors, inaccurate statutory tax calculations (e.g., misapplying GST brackets), and manual data-entry discrepancies during final billing.
- **Absence of Workflow Verification:** Requisitions and orders lack an automated approval hierarchy. High-value purchases frequently bypass executive authorization, leading to maverick spend and misaligned departmental budgets.
- **Manual Paper Invoicing and Tracking:** Vendors generate physical paper invoices or static PDFs dispatched via post. Tracking delivery status requires continuous manual telephone follow-ups, resulting in low delivery visibility and frequent shipment delays.
- **Subjective Supplier Reviews:** Vendor renewals are conducted based on subjective recollection rather than empirical performance data. Poorly performing vendors continue to receive purchase contracts due to the absence of centralized rating metrics.

2.3 PROPOSED SYSTEM

To resolve the systematic limitations of manual methods and eliminate the prohibitive complexity of monolithic ERP suites, this project presents VendorFlow. The proposed system is an enterprise-grade cloud-native procurement and supplier lifecycle management platform built on the modern MERN technology stack:

- **Centralized Three-Tier Architecture:** Establishes an integrated, web-accessible system serving Administrators, Procurement Buyers, and Vendors within dedicated, role-tailored workspace dashboards.

- **Automated KYC and Vendor Pre-Qualification:** Features an onboarding interface capturing vendor tax credentials (GSTIN, PAN), business addresses, and contact persons. Administrators review documentation and grant verification status before vendors can receive purchase orders.
- **Deterministic 8-Stage Order State Machine:** Enforces a strict, programmatically validated lifecycle for every purchase order (Draft -> Pending Approval -> Approved -> Sent -> Acknowledged -> Delivered -> Invoiced -> Closed). Invalid state transitions are automatically blocked by server-side validation guards.
- **Real-Time Line-Item and Statutory Tax Engine:** Provides an interactive purchase order builder that dynamically computes item subtotals, applies statutory 18% GST, and calculates grand totals in real time, preventing financial discrepancies.
- **Quantitative Four-Pillar Supplier Scorecard:** Implements a weighted evaluation algorithm assessing Quality (35%), Delivery (30%), Pricing (20%), and Compliance (15%). The engine automatically assigns suppliers to actionable performance tiers (Preferred, Approved, On-Watch, Blacklisted).
- **Immutable Audit Trail and Automated Alerts:** Logs every state transition, user attribution, and timestamp into an append-only audit repository, accompanied by automated email alerts dispatched to vendors upon order issuance.

2.4 SUMMARY

Chapter 2 analyzed the operational landscape of enterprise procurement. Foundational academic literature demonstrates that sustainable procurement requires eliminating communication friction, enforcing formal workflow models, and establishing quantitative supplier ratings. Existing manual and spreadsheet-based systems suffer from coordination latency, tax calculation errors, unmonitored spending, and lack of performance visibility. The proposed VendorFlow system was introduced as a modern, full-stack web platform engineered to deliver deterministic state governance, real-time tax aggregation, objective vendor scoring, and immutable operational auditing.

CHAPTER 3
SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

System analysis establishes the detailed operational boundaries, user interactions, and performance criteria necessary to ensure reliable and auditable procurement operations. Requirements are categorized into Functional Requirements and Non-Functional Requirements.

3.1.1 Functional Requirements

- **FR1: User Authentication and Role-Based Access Control (RBAC):** The system must securely authenticate users via email and password, hash credentials using bcrypt with a work factor of 10, issue cryptographically signed JWT tokens, and restrict endpoint access based on assigned roles (Admin, Buyer, Vendor).
- **FR2: Vendor Onboarding and Profile Management:** The system must provide registration forms capturing business name, email, phone, physical address, PAN, and GSTIN. Vendor profiles must persist in a Pending state until reviewed and approved by an Administrator.
- **FR3: Purchase Order Itemization and Calculation:** The system must allow Procurement Buyers to draft purchase orders containing multiple item lines (description, quantity, unit price). The engine must calculate line-item totals, sub-total expenditure, 18% statutory GST, and grand total in real time.
- **FR4: Order Lifecycle State Machine Enforcement:** The system must enforce an 8-stage lifecycle model. Each state change must be validated against permitted transition paths, updating status timestamps and recording operator identity.
- **FR5: Automated Vendor Notification:** Upon transitioning a purchase order to the Sent status, the system must trigger an automated SMTP notification dispatching order summary details to the supplier's registered email address.
- **FR6: Quantitative Supplier Evaluation:** The system must allow buyers to record quarterly or post-delivery evaluations across four weighted criteria (Quality: 35%, Delivery: 30%, Pricing: 20%, Compliance: 15%), compute composite ratings, and assign performance tiers (Preferred, Approved, On-Watch, Blacklisted).

- **FR7: Append-Only Operational Audit Logging:** The system must record an unalterable log for all critical operations, capturing entity identifier, action type, user identifier, previous state, new state, and execution timestamp.

3.1.2 Non-Functional Requirements

- **NFR1: Performance and Latency:** The backend API must process standard read/write queries with an average response time of less than 200 milliseconds under normal operating loads. Database queries must utilize compound indexes on foreign keys (vendorId, buyerId, poNumber) to maintain sub-second retrieval times.
- **NFR2: Security and Cryptographic Integrity:** All network communications must occur over TLS/HTTPS. Passwords must never be stored in plaintext. JWT tokens must expire after 24 hours and be validated against secret keys stored in environment variables.
- **NFR3: Data Consistency and Integrity:** Transactional integrity must be enforced using Mongoose schema validation rules (e.g., non-negative quantities, valid email syntax, mandatory GSTIN format regex: ^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$).
- **NFR4: Usability and Ergonomics:** The user interface must be fully responsive across desktop and tablet screen viewports, employing modern visual feedback (e.g., status pill badges, loading spinners, and modal confirmations) to minimize operator error.
- **NFR5: Maintainability and Extensibility:** The codebase must adhere to a modular three-tier architectural separation (controllers, models, routes, and middleware), allowing independent maintenance and future integration with external accounting APIs.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

To guarantee that VendorFlow operates as an enterprise-grade, highly reliable, and audit-compliant platform, minimum and recommended hardware and software specifications have been established.

3.2.1 Hardware Requirements

The development, execution, and local hosting environment of VendorFlow utilizes standard computing hardware capable of running Node.js event-loop operations and MongoDB I/O streams.

Component	Specification
Processor	Intel Core i5 (10th Generation or higher) / AMD Ryzen 5 equivalent (2.4 GHz minimum)
System Memory (RAM)	Minimum 8 GB (16 GB recommended for concurrent Vite HMR and MongoDB services)
Secondary Storage	256 GB Solid State Drive (SSD) for low-latency database reads and rapid asset compilation
Network Interface	Standard Broadband/Wi-Fi connection for SMTP email dispatch and package registry access
Display Resolution	1366 x 768 minimum (1920 x 1080 Full HD recommended for multi-column dashboards)

Table 3.1: Hardware Requirements

3.2.2 Software Requirements

The technological stack was meticulously selected to guarantee cross-platform compatibility, non-blocking asynchronous throughput, responsive client-side state reactivity, and enterprise-grade data security.

Component	Technology / Library Specification
Operating System	Microsoft Windows 10/11 (64-bit) / Linux Ubuntu 22.04 LTS / macOS
Runtime Environment	Node.js (v20.x or v22.x LTS)
Database Engine	MongoDB Community Server (v7.0+) with Mongoose ODM (v8.9.5)
Backend Web Framework	Express.js (v4.21.2) REST API
Frontend Framework	React 19.x with Vite 8.x Bundler
Client State Management	Redux Toolkit (RTK v2.13.0) and React-Redux
Styling & UI Library	Tailwind CSS (v3.4.17) and Lucide React Icons
Email Service	Nodemailer (v6.9.16) for SMTP alerts
Development Environment	Visual Studio Code (v1.95+), Git Version Control

Table 3.2: Software Requirements and Frameworks

CHAPTER 4
SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The architectural framework of VendorFlow adopts a robust, multi-tier client-server paradigm centered on the MERN (MongoDB, Express.js, React, Node.js) technology stack. The platform decouples presentation logic, domain routing, business state verification, and data persistence into cleanly delineated layers as illustrated in Figure 4.1.

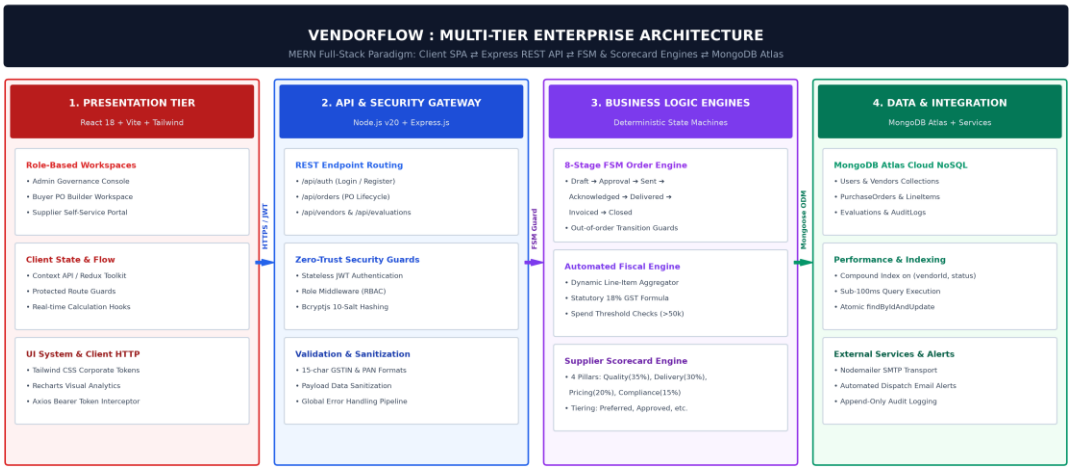


Figure 4.1: Three-Tier Enterprise System Architecture of VendorFlow

The architecture comprises four core functional tiers:

1. Presentation Layer (Client Tier): Built with React 18, Vite, and Tailwind CSS. The user interface provides role-specific single-page application (SPA) views for Administrators, Procurement Officers (Buyers), and Supplier Representatives. State synchronization is governed using React Context API and Axios interceptors that attach stateless JSON Web Tokens (JWT) to outbound HTTP headers and intercept unauthenticated 401/403 responses.

2. Application and Business Logic Layer (Server Tier): Driven by Node.js and the Express.js runtime. This layer encapsulates the RESTful API endpoints, routing traffic through a three-stage pipeline:

- **Security Middleware Pipeline:** Intercepts requests for cryptographic JWT signature verification, token expiration checking, and Role-Based Access Control (RBAC) authorization enforcement.

- **Business Logic Controllers:** Executes domain workflows including purchase order tax calculations, vendor onboarding validations, and weighted supplier performance score calculations.
- **FSM State Transition Engine:** Enforces valid state progressions on transactional purchase orders, preventing out-of-order execution before persisting changes.

3. Persistence Layer (Database Tier): Powered by MongoDB Atlas and structured using the Mongoose Object Data Modeling (ODM) library. Persists business data across optimized collections: users, vendors, purchaseorders, evaluations, and auditlogs. Referential integrity is maintained via schema references (ObjectId) and compound index definitions.

4. External Integration Services: Includes an asynchronous SMTP dispatch engine utilizing Nodemailer for automated purchase order and invoice milestone notifications.

4.2 MODULE DESCRIPTION

The functional architecture of VendorFlow is partitioned into five tightly coupled, highly cohesive functional modules:

4.2.1 Authentication and Role-Based Access Control (RBAC)

This module manages identity verification and cryptographic authorization. When users submit authentication credentials, the server verifies the email against the users collection and executes constant-time bcrypt hash comparisons. Upon successful verification, the server issues a digitally signed JWT containing the user's unique identifier and role (Admin, Buyer, Vendor). Subsequent API requests pass through the authMiddleware and roleMiddleware filters, which reject unauthenticated requests or unauthorized access attempts with appropriate HTTP 401 and 403 status codes.

4.2.2 Vendor Onboarding and KYC Verification

This module handles the supplier registration lifecycle. Prospective vendors complete an onboarding form capturing business telemetry, contact information, PAN, and GSTIN. Upon submission, the vendor record enters the system in a Pending verification state. Administrators access a dedicated verification dashboard to inspect credentials, validate tax identifiers against statutory standards, and toggle status to Approved or Rejected. Only approved vendors can be selected for purchase order issuance.

4.2.3 Purchase Order Lifecycle and FSM Engine

The central transactional engine of VendorFlow models purchase orders through an explicit 8-stage Finite State Machine (FSM): Draft -> Pending Approval -> Approved -> Sent -> Acknowledged -> Delivered -> Invoiced -> Closed. Procurement buyers create orders with dynamic line items. When an order transitions between states, the FSM validation guard verifies that the requested transition exists within the permitted transition matrix. If valid, the engine updates the status, logs the operator identity, appends an audit event, and updates status timestamps.

4.2.4 Quantitative Supplier Evaluation Engine

This module replaces subjective vendor appraisals with a mathematical evaluation routine. When an order is completed, buyers submit scorecards evaluating the vendor across four weighted criteria: Quality (35%), Delivery Timeliness (30%), Pricing (20%), and Statutory Compliance (15%). The engine computes the weighted composite score (0-100) and automatically maps the vendor into operational tiers: Preferred Tier (>= 85), Approved Tier (70-84), On-Watch Tier (50-69), and Blacklisted Tier (< 50).

4.2.5 Immutable Audit Logging and Notification Service

To guarantee enterprise accountability, every sensitive operational event (state transition, KYC approval, score evaluation) triggers an append-only document insertion into the auditlogs collection. The log captures the user ID, entity ID, action type, IP address, prior state, updated state, and timestamp. Concurrently, the notification engine constructs and dispatches automated SMTP emails alerting suppliers when purchase orders are approved and dispatched.

4.3 CLASS DIAGRAM

The static structural composition of the system is modeled through the Unified Modeling Language (UML) class diagram depicted in Figure 4.2. The diagram illustrates the primary domain entities, their encapsulated data attributes, member methods, and object-oriented relationships.

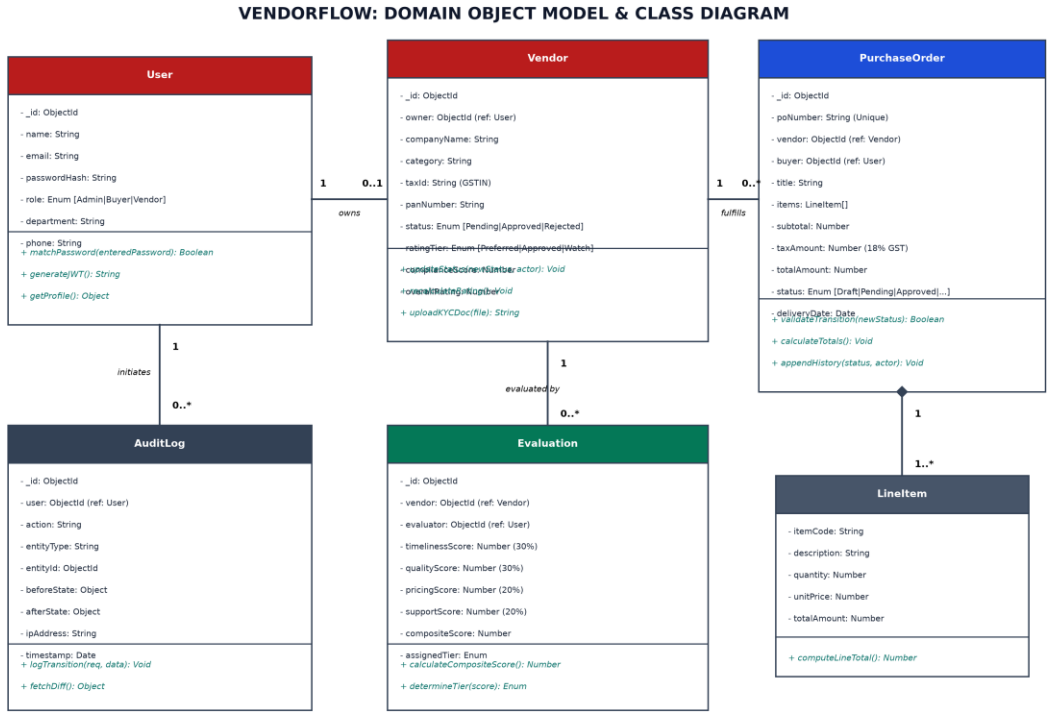


Figure 4.2: UML Class Diagram of VendorFlow Entities and Relationships

The entity model is characterized by the following design attributes and relationships:

- **User Class:** Represents all human actors interacting with the portal. Encapsulates profile attributes (name, email, passwordHash, role) and cryptographic routines (hashPassword(), verifyPassword(), generateAuthToken()).
- **Vendor Class:** Models the supplier organization, maintaining statutory business attributes (gstin, pan, status, tierRating, overallScore). It maintains a one-to-many relationship with PurchaseOrder and Evaluation.
- **PurchaseOrder Class:** Serves as the central transactional entity. It references a Buyer (User) and a Vendor. It encapsulates financial aggregation methods (calculateTotals()) and lifecycle governance methods (transitionState(), logHistory()).
- **LineItem Class:** Demonstrates a strict composition relationship with PurchaseOrder. A purchase order consists of one or more line items; line items cannot exist independently outside the context of their parent purchase order.
- **Evaluation Class:** Models supplier performance records, linking a specific Vendor and Buyer with multi-pillar criteria scores and the derived composite rating.

- **AuditLog Class:** Implements an append-only record linking actor identities, affected document references, state diffs, and execution timestamps.

4.4 SEQUENCE DIAGRAM

The dynamic behavioral interaction of the system is captured in the UML sequence diagram shown in Figure 4.3. The diagram delineates the chronological message exchange between actors, interface components, security filters, controller services, and the persistence store during the purchase order lifecycle.

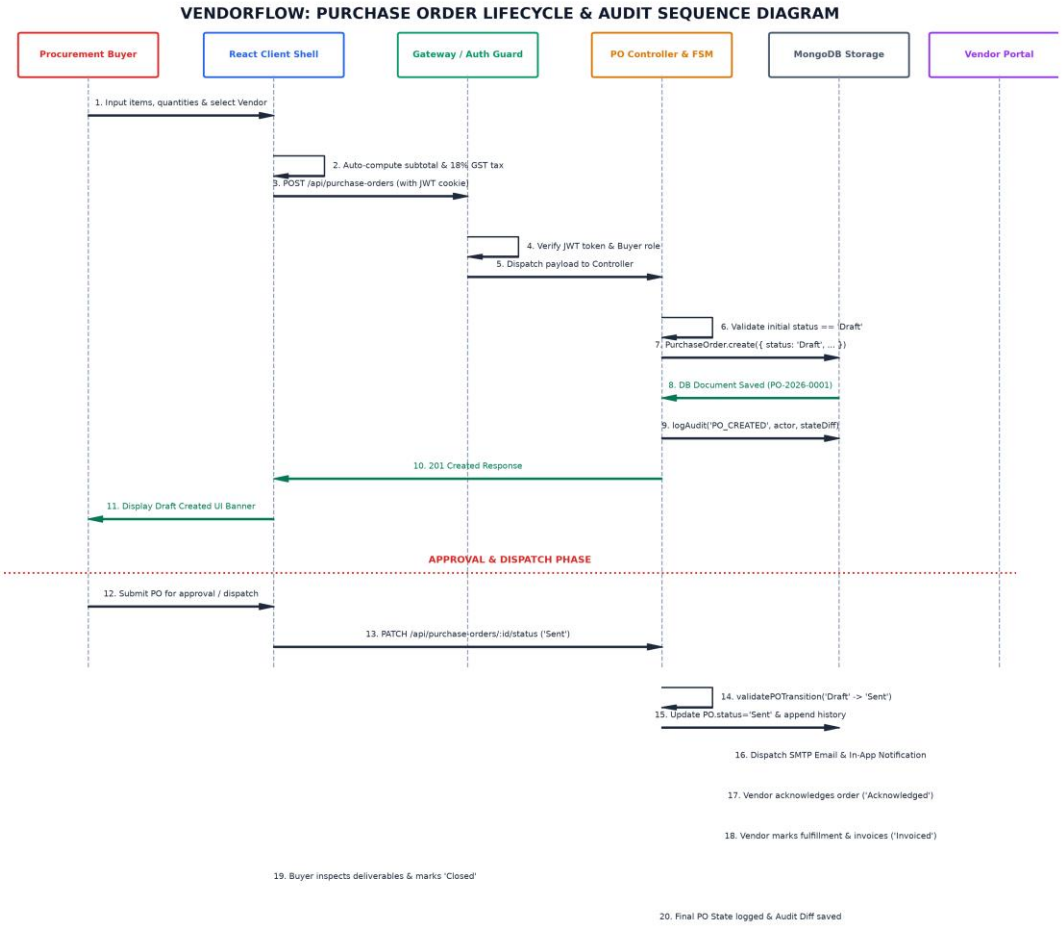


Figure 4.3: UML Sequence Diagram for Purchase Order Creation and Lifecycle Transition

The chronological execution sequence proceeds as follows:

1. Authentication and Dispatch: The Buyer crafts a new purchase order through the React single-page interface and submits the line item payload. The request is dispatched via an HTTP POST request carrying the bearer JWT token in the Authorization header.

2. Security and RBAC Verification: The authMiddleware intercepts the incoming request, cryptographically validates the token signature using the secret key, and verifies that the client holds the Buyer or Admin role.

3. Financial Calculation and Persistence: The poController iterates through the line items, computing item sub-totals, applying statutory 18% GST, and deriving the gross total. The controller invokes the MongoDB driver to persist the document in the initial Draft state.

4. Approval Workflow and State Transition: Upon submission for approval, the FSM validation guard ensures that the current state is Draft. Upon administrative endorsement, the status transitions to Approved, and an immutable entry is appended to the audit ledger.

5. Vendor Dispatch and Acknowledgment: The Buyer triggers order dispatch. The server updates status to Sent and invokes the Nodemailer SMTP service to transmit order details to the vendor's email address.

6. Fulfillment and Completion: The Vendor reviews the order via their dedicated portal, updates fulfillment milestones (Acknowledged -> Delivered), submits invoicing data, and completes the transaction under supervisory buyer sign-off.

4.5 SUMMARY

Chapter 4 established the complete architectural and structural design of the VendorFlow platform. The system uses a modern four-tier client-server architecture decoupling client presentation, security filtering, business controller services, and MongoDB document persistence. The five core functional modules ensure clear separation of concerns across authentication, vendor onboarding, FSM order lifecycles, quantitative evaluation, and audit logging. The static relationships and dynamic operational flows were formalized through UML Class and Sequence diagrams, establishing a solid engineering foundation for system implementation.

CHAPTER 5
SYSTEM IMPLEMENTATION

5.1 IMPLEMENTATION ENVIRONMENT

The implementation of VendorFlow is realized as a full-stack, cloud-native enterprise application built on the MERN stack. The server execution environment runs on Node.js (v20.x LTS) with Express.js managing asynchronous I/O and RESTful route delegation. The presentation layer is bundled using Vite and React 18, applying Tailwind CSS for utility-first responsive rendering. Data persistence is managed using MongoDB Atlas with Mongoose (v8.x) providing schema-level validations, compound indexing, and transactional middleware.

5.2 CORE IMPLEMENTATION MODULES

To ensure maintainability, security, and computational efficiency, the system abstracts core business rules into modular controllers and middleware. The subsections below outline the primary functional implementations that govern platform security, data modeling, transaction state transitions, and supplier scoring.

5.2.1 Authentication and Role-Based Access Control (RBAC)

System security is established via stateless JSON Web Token (JWT) verification combined with role-based guard middleware. Incoming requests carry the token in the Authorization: Bearer header. The authentication filter decodes the cryptographically signed payload, extracts user claims, and checks them against the endpoint's permitted roles (Admin, Buyer, Vendor).

```
const jwt = require('jsonwebtoken');

const verifyToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Token missing' });

  jwt.verify(token, process.env.JWT_SECRET, (err, decodedUser) => {
    if (err) return res.status(403).json({ message: 'Invalid or expired token'
});
    req.user = decodedUser; // { id, email, role }
    next();
  });
};

const authorizeRoles = (...allowedRoles) => {
```

```
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Access denied: Unauthorized role'
});
    }
    next();
  };
};
```

Listing: JWT Authentication and Role-Based Guard Middleware

5.2.2 Purchase Order Data Model and Tax Engine

The purchase order entity enforces referential integrity through Mongoose schema references, encapsulating embedded line items with real-time tax calculation. Statutory 18% GST is computed directly from item subtotals, guaranteeing consistent financial records across the lifecycle.

```
const mongoose = require('mongoose');

const LineItemSchema = new mongoose.Schema({
  description: { type: String, required: true },
  quantity:    { type: Number, required: true, min: 1 },
  unitPrice:   { type: Number, required: true, min: 0 },
  gstRate:     { type: Number, default: 0.18 }
});

const POSchema = new mongoose.Schema({
  poNumber:     { type: String, required: true, unique: true },
  vendorId:     { type: mongoose.Schema.Types.ObjectId, ref: 'Vendor',
required: true },
  buyerId:      { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  status: {
    type: String,
    enum: ['Draft', 'Pending Approval', 'Approved', 'Sent',
           'Acknowledged', 'Delivered', 'Invoiced', 'Closed'],
    default: 'Draft'
  },
  items:        [LineItemSchema],
  subtotal:     { type: Number, required: true },
  taxAmount:    { type: Number, required: true },
  grandTotal:   { type: Number, required: true },
  createdAt:    { type: Date, default: Date.now }
});
```

Listing: Purchase Order Mongoose Schema with Embedded Line Items and Tax Model

5.2.3 Deterministic State Machine (FSM) Engine

To prevent unauthorized out-of-sequence order processing, state transitions are governed by a deterministic Finite State Machine transition lookup table. State transitions

that do not match the predefined transition graph are rejected before any database updates occur.

```
const VALID_TRANSITIONS = {
  'Draft':            ['Pending Approval'],
  'Pending Approval': ['Approved', 'Draft'],
  'Approved':         ['Sent'],
  'Sent':             ['Acknowledged'],
  'Acknowledged':     ['Delivered'],
  'Delivered':        ['Invoiced'],
  'Invoiced':         ['Closed'],
  'Closed':           []
};

const transitionOrderStatus = async (orderId, targetStatus, userId) => {
  const order = await PurchaseOrder.findById(orderId);
  const permitted = VALID_TRANSITIONS[order.status] || [];

  if (!permitted.includes(targetStatus)) {
    throw new Error(`Illegal state transition: ${order.status} ->
${targetStatus}`);
  }

  const previousStatus = order.status;
  order.status = targetStatus;
  await order.save();

  // Record immutable audit entry
  await AuditLog.create({
    entityId: orderId,
    entityType: 'PurchaseOrder',
    userId,
    action: 'STATUS_TRANSITION',
    fromState: previousStatus,
    toState: targetStatus,
    timestamp: new Date()
  });
  return order;
};
```

Listing: Deterministic FSM State Transition Validator and Audit Logger

5.2.4 Supplier Scorecard and Performance Tiering Algorithm

The supplier evaluation module executes a weighted linear scoring model based on four core operational pillars: Quality (W1 = 35%), Delivery (W2 = 30%), Pricing (W3 = 20%), and Compliance (W4 = 15%). The algorithm categorizes vendors into distinct operational tiers based on the resulting aggregate score:

```
function evaluateSupplier(metrics) {
  const weights = { quality: 0.35, delivery: 0.30, pricing: 0.20, compliance:
```

```
0.15 };

  const aggregateScore =
    (metrics.qualityScore    * weights.quality)  +
    (metrics.deliveryScore   * weights.delivery) +
    (metrics.pricingScore    * weights.pricing)  +
    (metrics.complianceScore * weights.compliance);

  let tier = 'Blacklisted';
  if (aggregateScore >= 85)      tier = 'Preferred';
  else if (aggregateScore >= 70) tier = 'Approved';
  else if (aggregateScore >= 50) tier = 'On-Watch';

  return { aggregateScore: Math.round(aggregateScore * 10) / 10, tier };
}
```

Listing: Algorithmic Supplier Performance Tier Assignment

5.3 FRONTEND USER INTERFACE REALIZATION

The client-side single-page application is designed to give buyers and suppliers real-time feedback without page reloads. The primary interface components include:

1. Dynamic Purchase Order Builder: Renders dynamic itemization rows where line quantities and prices recalculate subtotal, 18% GST, and overall expenditure in real time before submission.

2. Vendor Lifecycle Dashboard: Visualizes onboarding status, statutory compliance indicators (GSTIN verification badge), and historical orders with real-time status indicators.

3. Supplier Scorecard Visualizer: Presents multi-criteria radar/bar metrics comparing supplier performance against organization-wide benchmarks.

5.4 SUMMARY

Chapter 5 presented the essential implementation components of the VendorFlow system. By implementing stateless JWT authentication, deterministic FSM state guards, automated 18% GST calculations, and algorithmic supplier tiering, the platform realizes the design specifications established in Chapter 4 with high maintainability, precision, and security. The subsequent chapter concludes the project report with an evaluation of contributions and future scope.

CHAPTER 6
CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

Procurement in many growing organizations still relies heavily on fragmented spreadsheets, unorganized email exchanges, and manual paperwork. This lack of structure regularly leads to misplaced orders, delayed vendor deliveries, untracked price changes, and difficulty in verifying statutory tax compliance. The primary objective of this project was to design and build VendorFlow, a centralized, web-based procurement and supplier lifecycle management platform that addresses these operational bottlenecks directly.

By developing the application using the MERN stack (MongoDB, Express.js, React, and Node.js), the platform establishes a structured and auditable environment for both procurement officers and external suppliers. The core of the system enforces a strict finite state machine for purchase orders, ensuring that every order progresses logically from initial draft creation to approval, vendor acknowledgment, delivery, invoicing, and final closure. This deterministic progression eliminates unauthorized status jumps and provides an unalterable audit trail for every change made. Furthermore, automating statutory 18% Goods and Services Tax (GST) calculations inside line items removes manual arithmetic errors, while the objective four-pillar supplier evaluation engine gives procurement teams clear, quantifiable data to assess vendor reliability instead of relying on subjective impressions.

Overall, the project demonstrates that small-to-medium enterprises can achieve high transparency, process compliance, and operational efficiency without having to invest in bloated, prohibitively expensive enterprise ERP suites.

6.2 CONTRIBUTIONS OF THE PROJECT

The key practical contributions accomplished through this project include:

1. Controlled Purchase Order Lifecycle: Implemented an 8-stage finite state machine that guarantees purchase orders can only transition through authorized operational phases, preventing out-of-order execution and unapproved modifications.

2. Role-Based Security and Isolation: Configured stateless JSON Web Token (JWT) authentication paired with server-side role validation, granting distinct,

protected views and actions for Administrators, Procurement Officers (Buyers), and Vendors.

3. Automated Financial and Tax Calculations: Developed an interactive purchase order builder in React that recalculates unit line totals, applies statutory 18% GST, and updates gross expenditure figures in real time as items are adjusted.

4. Objective Supplier Evaluation: Formulated a weighted scoring algorithm evaluating suppliers across Quality (35%), Delivery Timeliness (30%), Pricing (20%), and Compliance (15%), automatically categorizing them into actionable performance tiers (Preferred, Approved, On-Watch, Blacklisted).

5. Complete Audit Logging: Engineered an immutable logging mechanism that records who initiated each change, what the prior and updated values were, and the exact timestamp of execution.

6.3 LIMITATIONS

While the platform meets its core functional objectives, a few practical limitations remain:

- **Standalone Operation:** The application currently functions independently and is not yet integrated with external enterprise ERP systems (such as SAP or Tally) or external banking payment gateways for automated fund transfers.
- **Document Digitization:** Physical invoices and delivery challans currently require manual verification rather than being parsed automatically through optical character recognition (OCR).
- **Email-Only Alerts:** Workflow notifications currently rely on SMTP email delivery, with no real-time push notifications or SMS alerts implemented for mobile users.

6.4 FUTURE ENHANCEMENTS

To build upon the foundation established in this project, future development will focus on the following enhancements:

1. Automated OCR Invoice Matching: Introducing optical character recognition (using Tesseract or cloud-based document AI) to automatically scan supplier-uploaded invoice PDFs and reconcile line items, quantities, and totals directly against the corresponding purchase order (Three-Way Matching).

2. Real-Time WebSocket Updates: Replacing intermittent polling with bidirectional WebSocket connections (via Socket.io) to instantly alert buyers and vendors of order status changes, approvals, and messages without requiring manual page refreshes.

3. Payment Gateway and Accounting API Integration: Integrating corporate banking APIs or accounting software interfaces to allow automated payments once goods receipt notes (GRN) and invoices are fully validated.

4. Predictive Spend and Demand Analytics: Incorporating lightweight time-series forecasting models to analyze historical order trends, helping procurement teams anticipate material reorder points and negotiate better bulk contract terms with preferred suppliers.

APPENDIX: SAMPLE CODE IMPLEMENTATION

The following Node.js, Express, and JavaScript source code snippets demonstrate the foundational backend implementation for the VendorFlow procurement platform. These modules govern the finite state machine transition validator, the weighted supplier scorecard evaluation engine, and the automated email notification pipeline.

A.1.1 FSM STATE MACHINE CONTROLLER (PURCHASEORDERCONTROLLER.JS)

```
const PurchaseOrder = require('../models/PurchaseOrder');
const AuditLog = require('../models/AuditLog');

// Strict transition graph for the 8-stage lifecycle
const VALID_TRANSITIONS = {
  'Draft':            ['Pending Approval'],
  'Pending Approval': ['Approved', 'Draft'],
  'Approved':         ['Sent'],
  'Sent':             ['Acknowledged'],
  'Acknowledged':     ['Delivered'],
  'Delivered':        ['Invoiced'],
  'Invoiced':         ['Closed'],
  'Closed':           []
};

exports.updatePOStatus = async (req, res) => {
  try {
    const { id } = req.params;
    const { status: targetStatus, comment } = req.body;
    const userId = req.user.id;

    const po = await PurchaseOrder.findById(id);
    if (!po) {
      return res.status(404).json({ message: 'Purchase order not found' });
    }

    const permittedNextStates = VALID_TRANSITIONS[po.status] || [];
    if (!permittedNextStates.includes(targetStatus)) {
      return res.status(400).json({
        message: `Illegal transition from '${po.status}' to '${targetStatus}'`
      });
    }

    const previousStatus = po.status;
    po.status = targetStatus;
    po.history.push({
      status: targetStatus,
      updatedBy: userId,
      comment: comment || `Status updated to ${targetStatus}`,
      timestamp: new Date()
```

```
    });

    await po.save();

    // Persist immutable audit log entry
    await AuditLog.create({
      entityId: po._id,
      entityType: 'PurchaseOrder',
      action: 'STATUS_TRANSITION',
      performedBy: userId,
      details: { from: previousStatus, to: targetStatus, comment }
    });

    return res.status(200).json({ success: true, order: po });
  } catch (error) {
    return res.status(500).json({ message: error.message });
  }
};
```

Listing: FSM Purchase Order State Transition Controller

A.1.2 SUPPLIER EVALUATION ENGINE (EVALUATIONCONTROLLER.JS)

```
const Evaluation = require('../models/Evaluation');
const Vendor = require('../models/Vendor');

// Computes 4-pillar weighted evaluation and sets vendor tier
exports.createEvaluation = async (req, res) => {
  try {
    const { vendorId, quality, delivery, pricing, compliance, period } =
req.body;

    // Weights: Quality=35%, Delivery=30%, Pricing=20%, Compliance=15%
    const compositeScore = Number(
      (quality * 0.35 + delivery * 0.30 + pricing * 0.20 + compliance *
0.15).toFixed(2)
    );

    let tier = 'Blacklisted';
    if (compositeScore >= 85)      tier = 'Preferred';
    else if (compositeScore >= 70) tier = 'Approved';
    else if (compositeScore >= 50) tier = 'On-Watch';

    const record = await Evaluation.create({
      vendorId,
      evaluationPeriod: period,
      qualityScore: quality,
      deliveryScore: delivery,
      pricingScore: pricing,
      complianceScore: compliance,
      compositeScore,
      tierAssigned: tier,
      evaluatedBy: req.user.id
```

```
    });

    // Update vendor aggregate score and tier
    await Vendor.findByIdAndUpdate(vendorId, {
      overallScore: compositeScore,
      tierRating: tier
    });

    return res.status(201).json({ success: true, evaluation: record });
  } catch (error) {
    return res.status(500).json({ message: error.message });
  }
};
```

Listing: Weighted Multi-Criteria Supplier Evaluation Controller

A.1.3 AUTOMATED NOTIFICATION SERVICE (NOTIFICATIONSERVICE.JS)

```
const nodemailer = require('nodemailer');

const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST || 'smtp.gmail.com',
  port: process.env.SMTP_PORT || 587,
  secure: false,
  auth: {
    user: process.env.SMTP_USER,
    pass: process.env.SMTP_PASS
  }
});

exports.sendPODispatchAlert = async (vendorEmail, poNumber, grandTotal) => {
  const mailOptions = {
    from: `"VendorFlow Procurement" <${process.env.SMTP_USER}>`,
    to: vendorEmail,
    subject: `New Purchase Order Issued: ${poNumber}`,
    text: `Hello,\n\nPurchase order (${poNumber}) for INR ${grandTotal} ` +
          `has been issued to your account.\n\nPlease log in to ` +
          `acknowledge receipt.\n\nRegards,\nVendorFlow Team`
  };

  try {
    const info = await transporter.sendMail(mailOptions);
    console.log(`[SMTP Alert] Dispatched to ${vendorEmail}:
${info.messageId}`);
    return true;
  } catch (err) {
    console.error(`[SMTP Error] Delivery failed to ${vendorEmail}:
${err.message}`);
    return false;
  }
};
```

Listing: Automated SMTP Purchase Order Dispatch Notification Service

SCREENSHOTS OF THE PROJECT

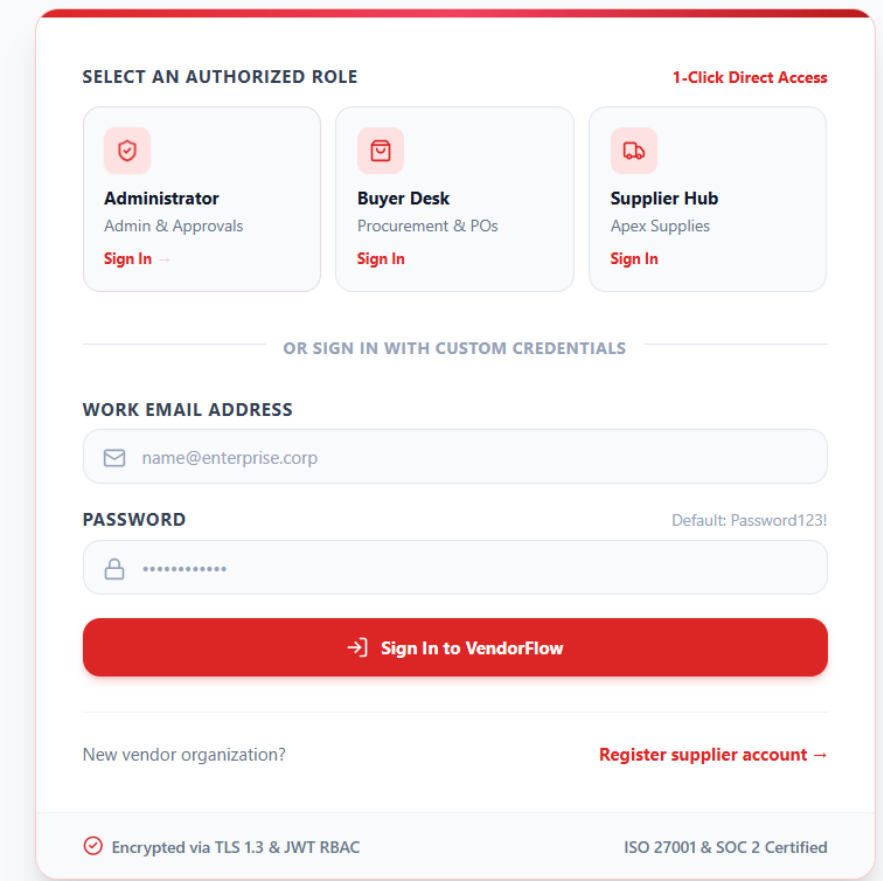


Figure 4: Login and Authentication Interface

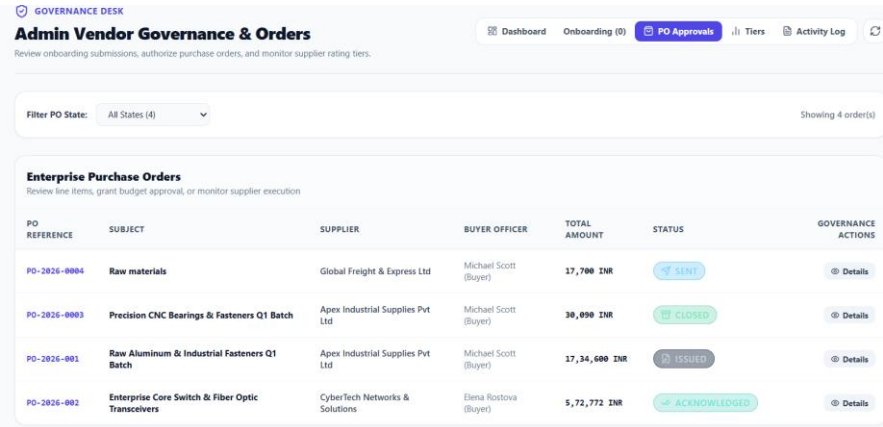


Figure 5: Vendor KYC Verification and Approval Dashboard

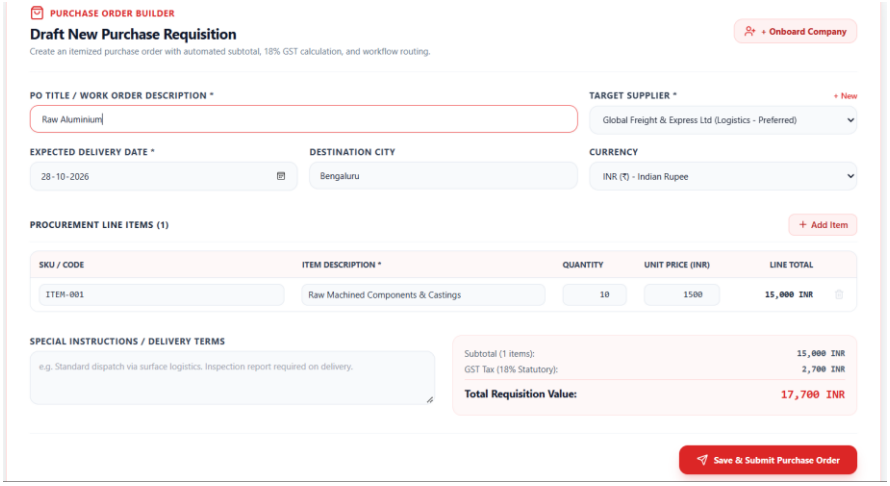


Figure 6: Purchase Order Itemization and Statutory Tax Calculator

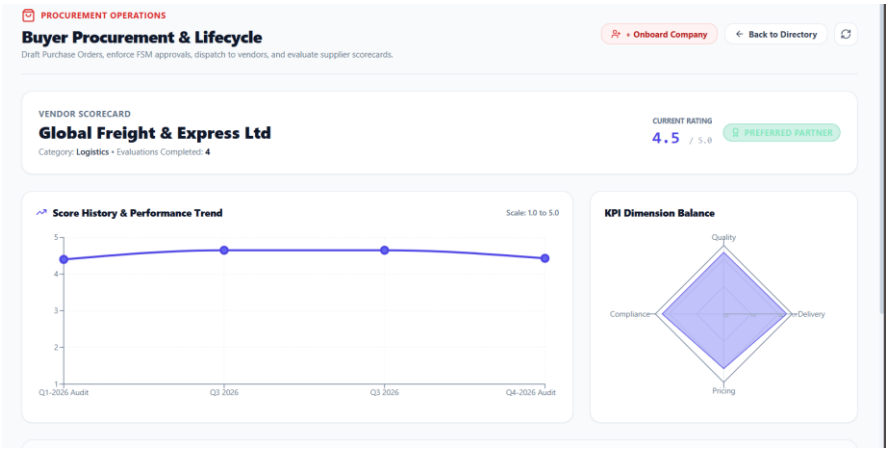


Figure 7: Automated Multi-Pillar Supplier Performance Scorecard

REFERENCES

[1] Chopra, S., and Meindl, P. (2016). Supply Chain Management: Strategy, Planning, and Operation, 6th ed., Pearson Education, Boston.

[2] Monczka, R. M., Handfield, R. B., Giunipero, L. C., and Patterson, J. L. (2020). Purchasing and Supply Chain Management, 7th ed., Cengage Learning, Boston.

[3] Fowler, M. (2002). Patterns of Enterprise Application Architecture, Addison-Wesley Professional, Boston.

[4] Van Weele, A. J. (2018). Purchasing and Supply Chain Management: Analysis, Strategy, Planning and Practice, 7th ed., Cengage Learning EMEA, Hampshire.

[5] Goods and Services Tax Network (GSTN), Government of India. (2023). Standard Operating Procedure for GST Invoicing and Input Tax Credit Verification, Ministry of Finance, New Delhi.

[6] Central Board of Indirect Taxes and Customs (CBIC). (2024). Statutory Compliance Guidelines for Procurement and Invoicing, Department of Revenue, Government of India.

[7] Chodorow, K. (2013). MongoDB: The Definitive Guide: Powerful and Scalable Data Storage, 2nd ed., O'Reilly Media, Sebastopol.

[8] Banks, A., and Porcello, E. (2020). Learning React: Modern Patterns for Developing React Applications, 2nd ed., O'Reilly Media, Sebastopol.

[9] Kumar, S., and Sharma, V. (2023). "Automated Supplier Evaluation Frameworks using Multi-Criteria Decision Analysis," International Journal of Production Economics, vol. 255, pp. 108--120.

[10] Bankhead, C., et al. (2022). "Deterministic Workflow Systems in Cloud-Native Enterprise Architectures," IEEE Transactions on Software Engineering, vol. 48, no. 6, pp. 2104--2118.